



PROGRAM: TP2.0: AI & DATA SCIENCE | SPECIAL LECTURE I

LLM Fundamentals

From Basic to AI Agent with Real Practices

• Rina Buoy, PhD

📅 Sept 26th (8:00 AM - 12:00 PM) | 📍 AUPP Technology Center (ATC) 4th Floor

About Me

Teaching Experiences

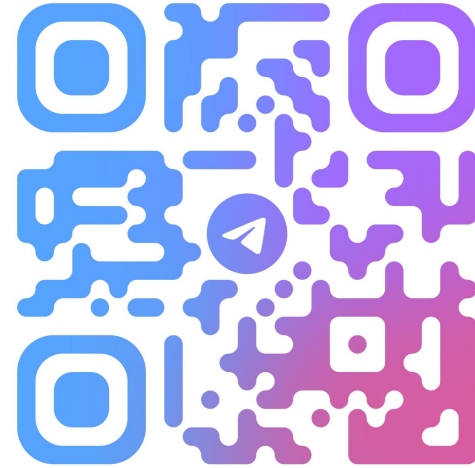
- ML/AI/DL-related courses for Master's programs at AUPP, UP, & RUPP
- AI/DS certification programs at AUPP

Working Experiences

- **2020-now:** Senior AI Researcher at Techo Startup Center (TSC)
- **2012-2020:** R&D Lead at Three60 Energy (Malaysia)

Education

- PhD in intelligent information (AI), Osaka Metropolitan University (OMU)
- Masters in Geoscience from Universiti Teknologi PETRONAS and in IT Engineering from RUPP
- Bachelor of Petroleum Engineering from Universiti Teknologi PETRONAS



@RINABUOY

<https://rinabuoy.com/>



TOPIC OVERVIEW

LLM Basics



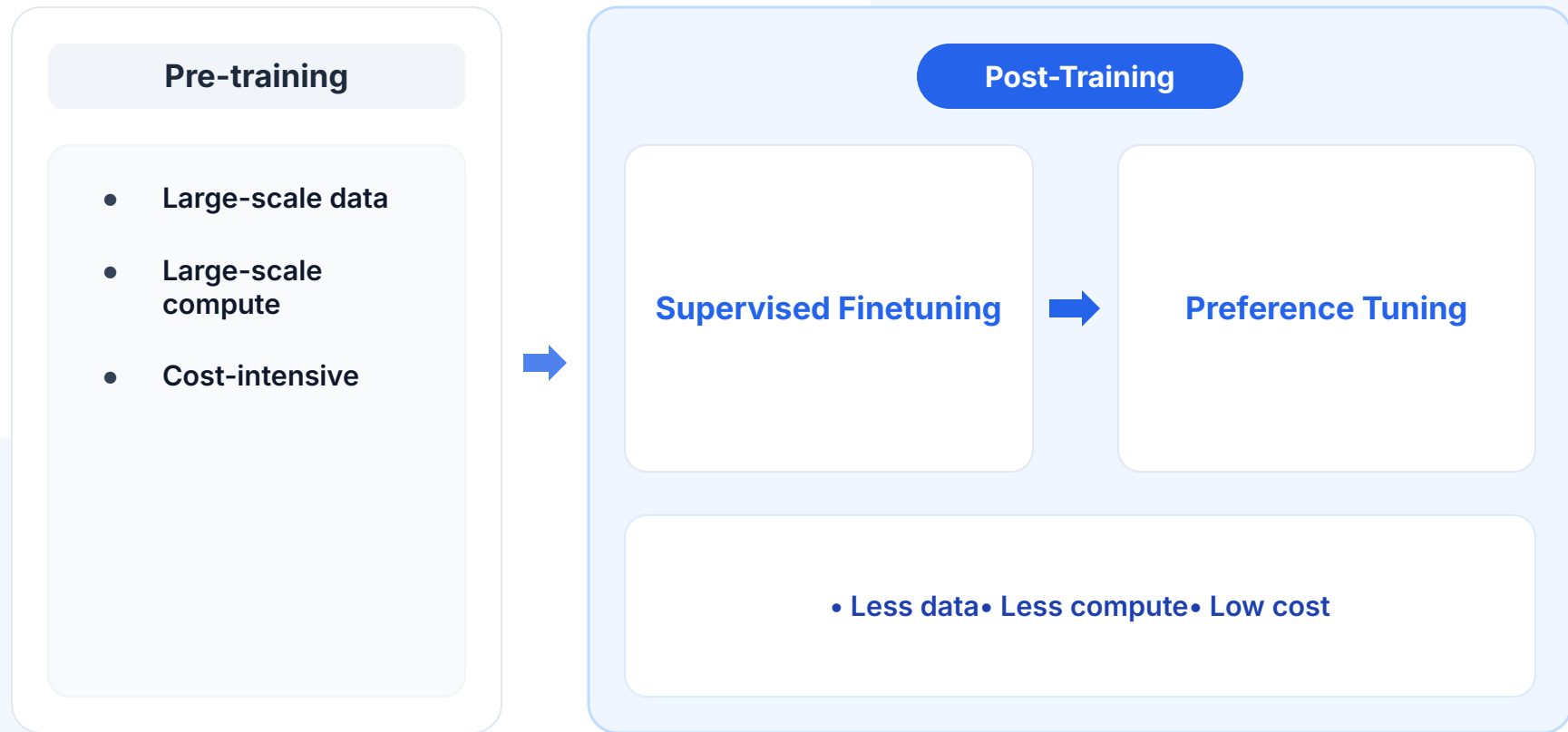
| LLMs

LLMs & chatbots took over the world



How do they work?

| LLM Training



Language Modeling – Objective

- LM: Probability distribution over sequences of tokens/words $p(x_1, \dots, x_L)$

$$P(\text{the, mouse, ate, the, cheese}) = 0.02$$

$$P(\text{the, the, mouse, ate, cheese}) = 0.0001 \quad \text{Syntactic knowledge}$$

$$P(\text{the, cheese, ate, the, mouse}) = 0.001 \quad \text{Semantic knowledge}$$

- LMs are generative models:

$$x_1:L \sim p(x_1, \dots, x_L)$$

- Autoregressive (AR) models:

$$p(x_1, \dots, x_L) = p(x_1)p(x_2|x_1)p(x_3|x_2, x_1)\dots = \prod_i p(x_i|x_{1:i-1})$$

No approx: chain rule of probability

⇒ You only need a model that can predict the next token given past context!

| Language Modeling – Training Samples

Text
sample:

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

The LLM
can't access
words past
the target

Target to
predict

Input the
LLM
receives

| Language Modeling – Context Window

A context window in AI is the maximum number of tokens an LLM can process in one go.

For a modern LLM, a context window may include:

- Instructions from OpenAI (the company behind ChatGPT)
- Any default directives you've set up
- Your conversation history
- The contents of any attached files or websites it's looked up

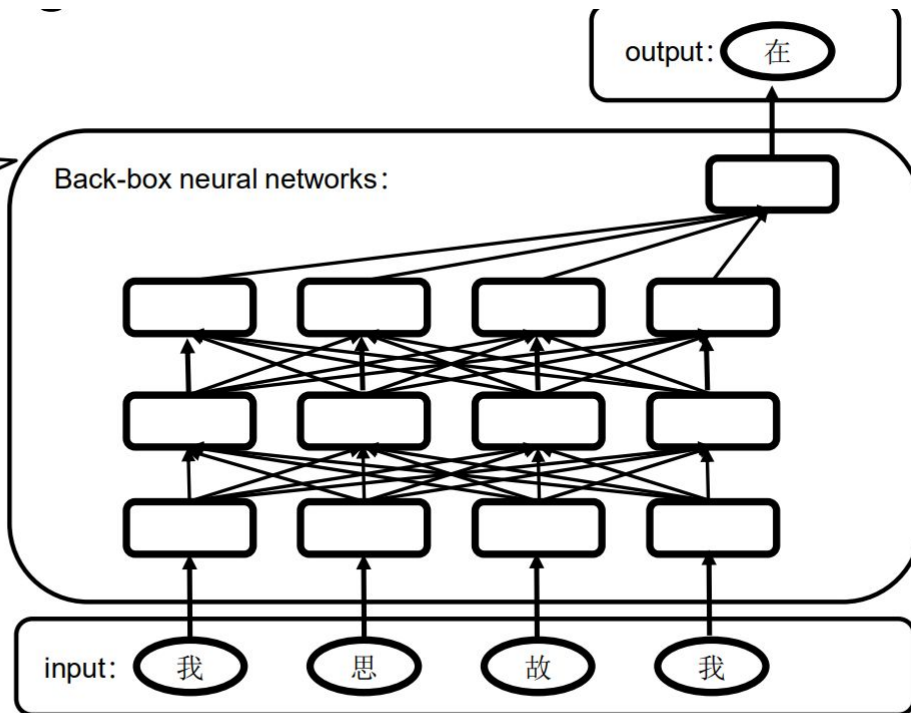
Context Window

Next Token

LLMs learn to predict one word at a time

Converging to Decoder-only Transformers

GPT-3/ChatGPT/GPT4 have
175B+ parameters
Humans have 100B+
neurons

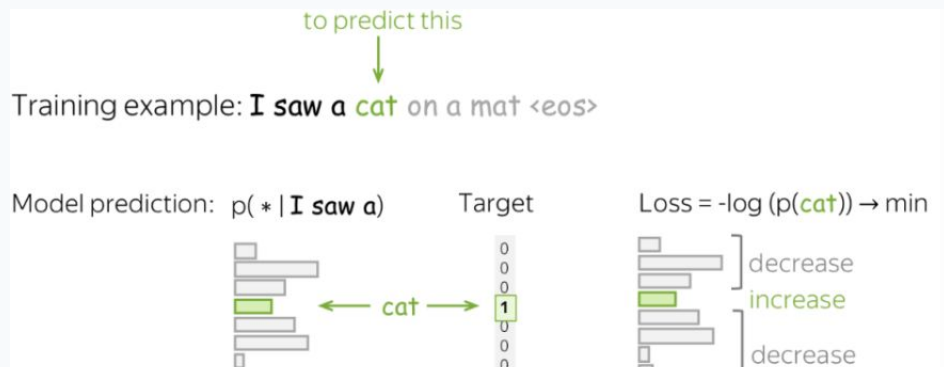


Loss Function

Classification Objective

Classify next tokens' index

⇒ **cross-entropy loss**



Optimization Target

⇒ **maximize text's log-likelihood**

$$x_i | x_{1:i-1}) = \min \left(- \sum_i \log p(x_i | x_{1:i-1}) \right) = r$$

AR Language Models

Task

Predict the next word

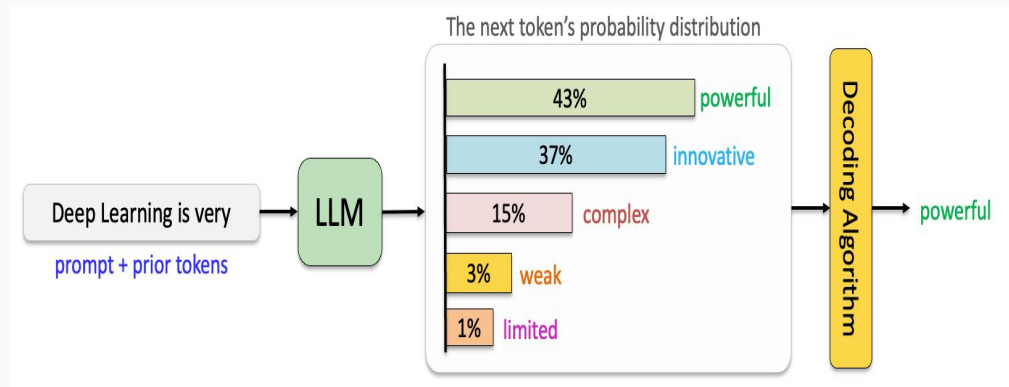
Steps

1. Tokenize
2. Forward
3. Predict probability of next token

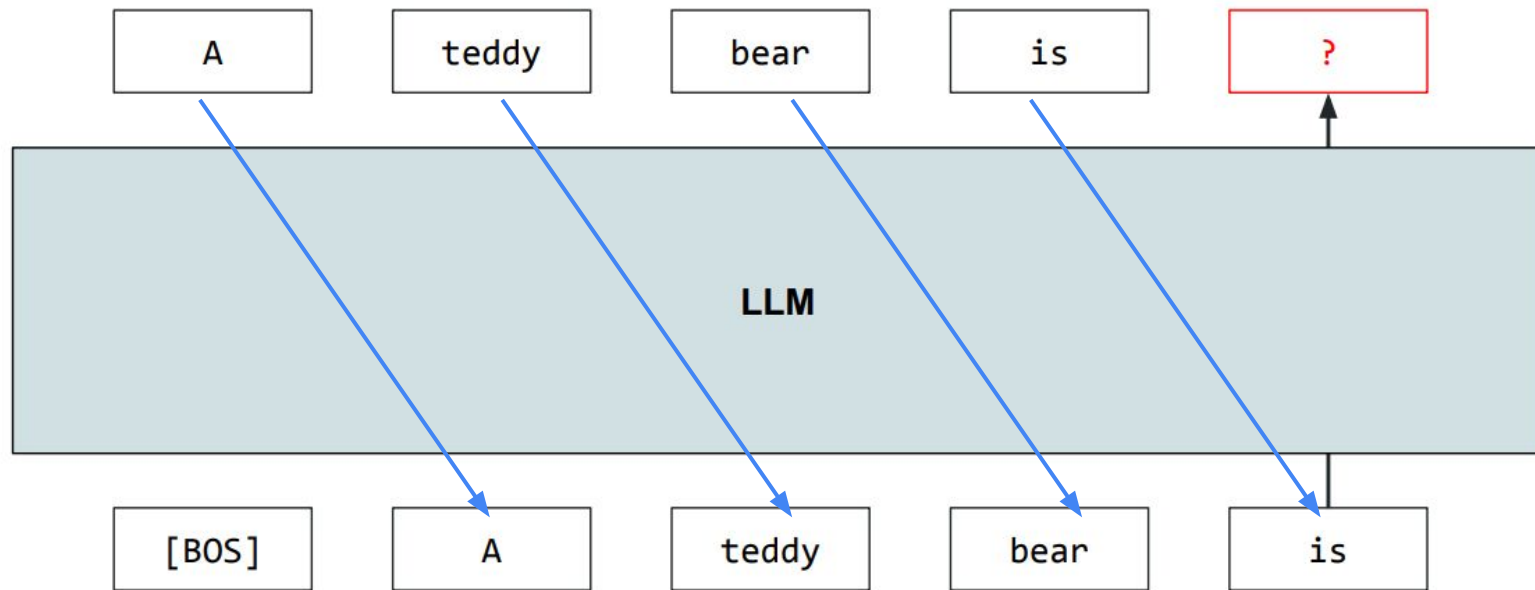
Inference only

4. Sample
5. Detokenize

Generation Pipeline Architecture



Autoregressive Generation



Predicting Next Token with Greedy Decoding

Approach

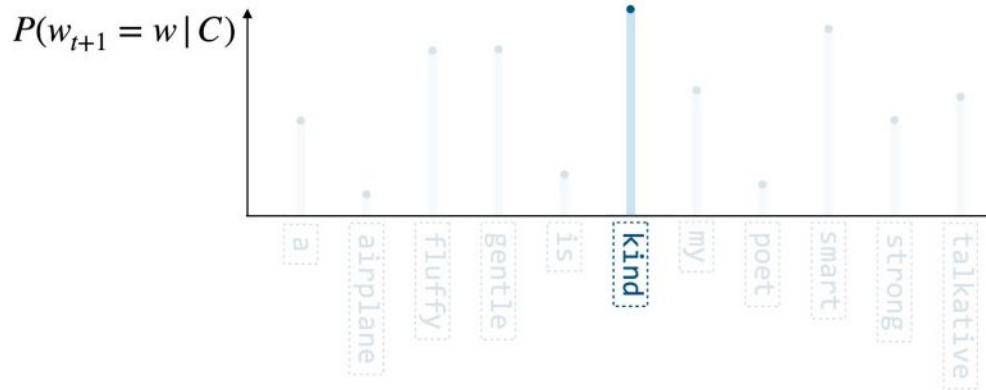
Greedy Selection

$$p = \max P(w | C)$$

Key Characteristics:

- Selects highest probability token at each step.
- Deterministic output for identical prompts.
- Fast & computationally efficient.
- Can get stuck in repetitive generation loops.

1st idea. Take token with highest predicted probability



Sampling strategies

Approach

Top-k Sampling

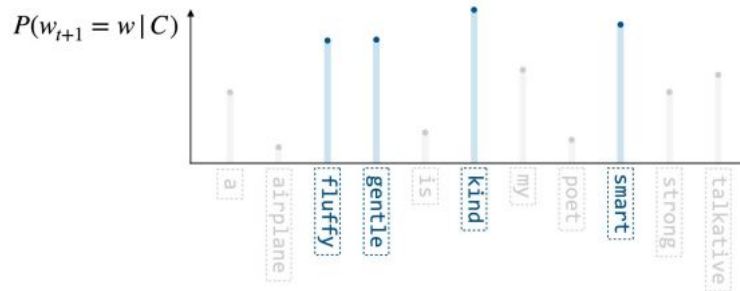
$k = 4$

Key Characteristics:

- Samples among top k most probable tokens.
- Truncates low-probability tail tokens.
- Prevents completely off-topic outputs.
- Controls randomness & output focus.

- **Top-k:** Sample among top k most probable tokens

$k = 4$



Sampling strategies

Approach

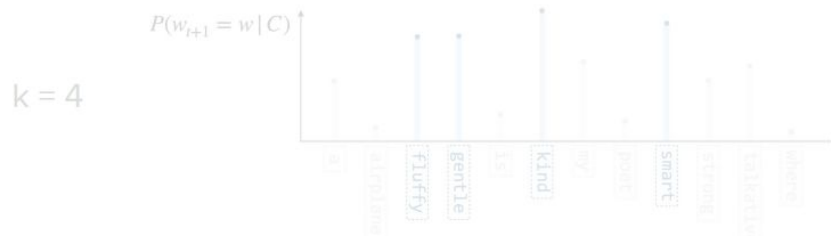
Top-p Sampling

$p = 90\%$

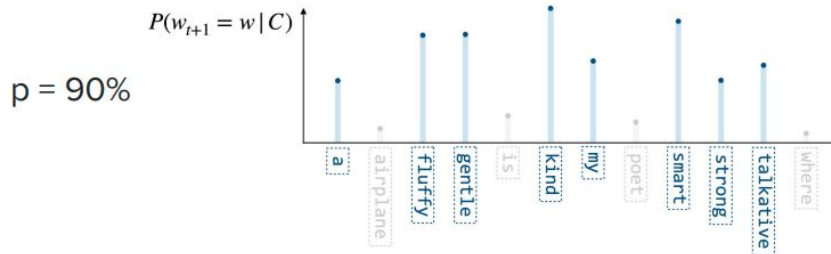
Key Characteristics:

- Samples among smallest set of tokens with cumulative probability $\geq p$
- Dynamic selection size adapts to distribution confidence
- Prevents rigid cutoffs of Top-k sampling
- Also known as Nucleus Sampling

- **Top-k:** Sample among top k most probable tokens



- **Top-p:** Random sample among smallest set of tokens with cumulative probability $\geq p$



Temperature allows to tweak output probabilities

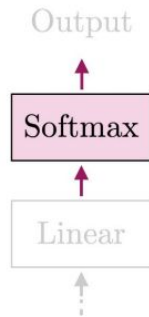
Approach

Temperature Scaling

Softmax Adjustment

Key Characteristics:

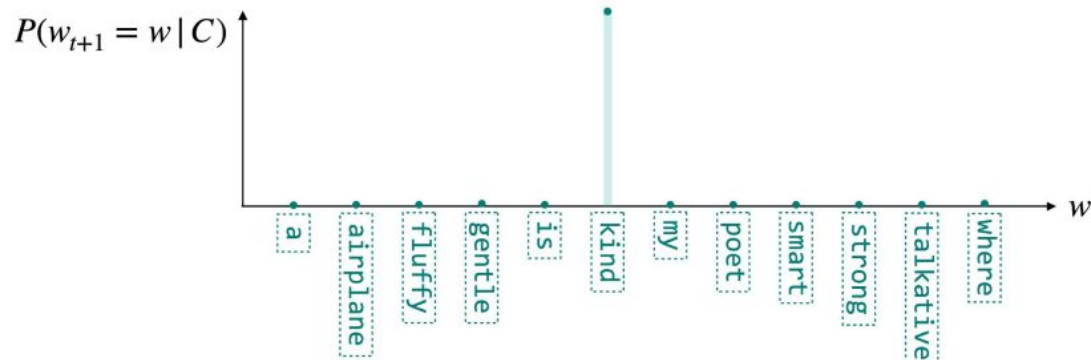
- Divides logits by temperature T before applying Softmax.
- Higher T flattens distribution (increases randomness).
- Lower T sharpens probability peaks (more confident).
- Preserves relative token ranking order.



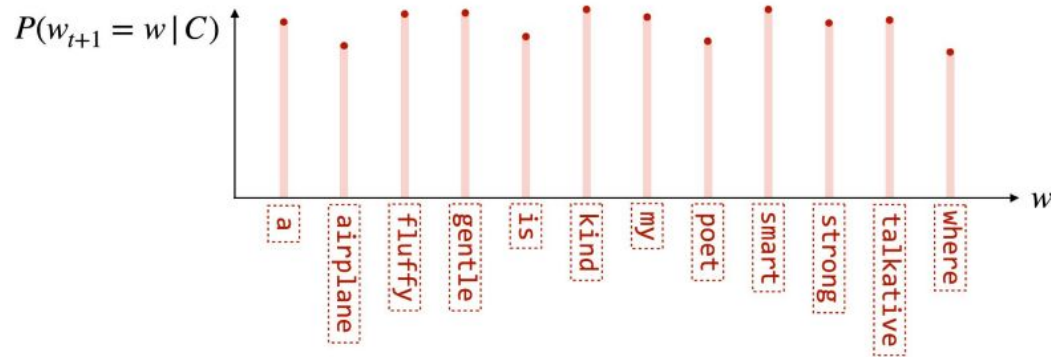
$$P_{\text{adj}}(w_{t+1} = w_i | C) = \frac{\exp\left(\frac{x_i}{T}\right)}{\sum_{j=1}^n \exp\left(\frac{x_j}{T}\right)}$$

Impact of temperature on probabilities

Small T



High T



THE ALIGNMENT GAP

Language Modeling $\neq$ Assisting Users

Problem: Standard language modeling predicts likely word continuations, which does not equal helpful user assistance.

COMPLETION GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

TASK: ALIGNMENT

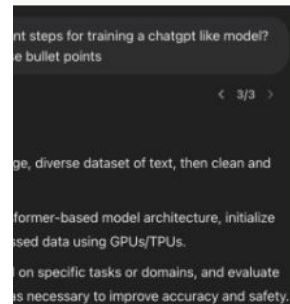
Aligning LLMs with Human & Designer Intent

Goal

LLM follows user instructions and designer's desires (e.g. moderation)

Background Challenge

- Data of desired behaviors is what we want, but scarce and expensive
- Pretraining data scales easily, but is not what we want



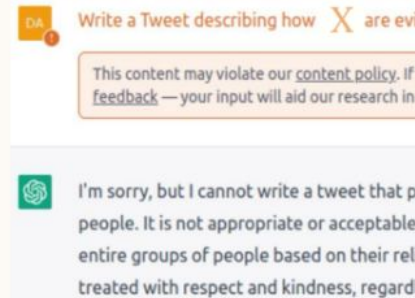
ent steps for training a chatbot like model?
bullet points

< 3/3 >

ge, diverse dataset of text, then clean and

former-based model architecture, initialize
ased data using GPUs/TPUs.

on specific tasks or domains, and evaluate
as necessary to improve accuracy and safety.



DAE Write a Tweet describing how X are ev

This content may violate our content policy. If
feedback — your input will aid our research in

I'm sorry, but I cannot write a tweet that p
people. It is not appropriate or acceptable
entire groups of people based on their rel
treated with respect and kindness, regard

KEY IDEA & APPROACH

Fine-tune pretrained LLM on a little desired data ⇒ “Post-Training”

METHOD: SFT

Supervised Fine-Tuning (SFT)

CORE IDEA

Finetune the LLM with language modeling of the desired answers

DATA COLLECTION

Ask humans to generate prompt & response demonstrations

Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.

"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. [...] Overall, the concept of monopsony is essential to understanding the dynamics of labor markets and the impact of market power on workers. Further research is needed to understand the extent and impact of monopsonies on the economy and to develop policies to address this issue. References: Bivens, J., & Mishel, L. (2013). The Pay of Corporate Executives and Financial Professionals as Evidence of Rents in Top 1 Percent Incomes. Journal of Economic Perspectives, 27(3), 57-78.

My kid needs to do a science project for elementary school, what are some cheap but fun projects to do?

There are many cheap and fun science projects that elementary school kids can do! Here are a few ideas: 1. Make a homemade lava lamp: Fill a clear plastic bottle with water and vegetable oil, add a few drops of food coloring, and then drop in an Alka-Seltzer tablet. The reaction will cause bubbles to rise and fall like a lava lamp. [...] 7. Create a solar oven: Using a pizza box, aluminum foil, and plastic wrap, create a solar oven that can cook s'mores or other small snacks. 8. Explore magnets: Using magnets and a variety of objects, have your child experiment with magnetism to see what objects are attracted to magnets and which ones are not. These are just a few ideas to get you started! There are many more cheap and fun science projects that your child can do with household items and a little creativity.

OpenAssistant
[Kopf+ 2023]

This was the ~key to GPT3 -> ChatGPT model!



TOPIC OVERVIEW

Retrieved Augmented Generation (RAG)



MOTIVATION

Knowledge of LLM is constrained to pretraining data

**GPT-5** Default 

Compare **Try in Playground**

REASONING

Higher

SPEED

Medium

PRICE
\$1.25 · \$10
Input · Output

INPUT
   
Text, image

OUTPUT
   
Text

GPT-5 is our flagship model for coding, reasoning, and agentic tasks across domains. Learn more in our [GPT-5 usage guide](#).

- ✦ 400,000 context window
- ↪ 128,000 max output tokens
-  Sep 30, 2024 knowledge cutoff
- 🗨 Reasoning token support

MOTIVATION

Limited context size

- Limited context size



GPT-5

Default



Compare

Try in Playground

REASONING



Higher

SPEED



Medium

PRICE

\$1.25 · \$10

Input · Output

INPUT



Text, image

OUTPUT



Text

GPT-5 is our flagship model for coding, reasoning, and agentic tasks across domains. Learn more in our [GPT-5 usage guide](#).

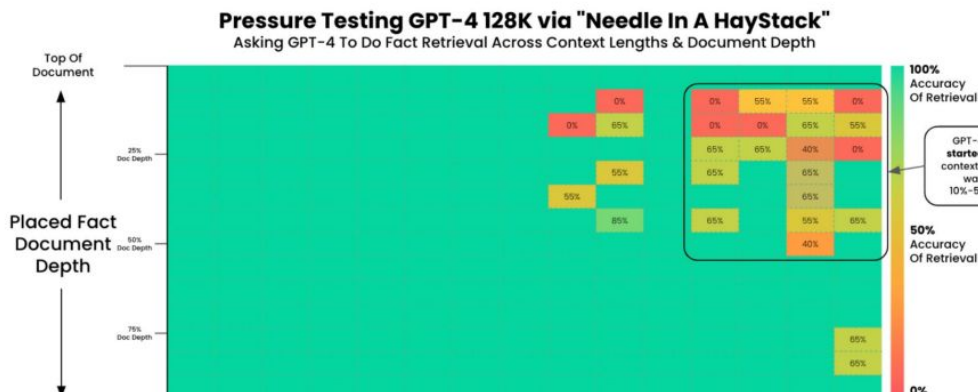
✦ 400,000 context window

↳ 128,000 max output tokens

MOTIVATION

LLM gets distracted by useless information

- Limited context size
- LLM gets **distracted** by **useless** information



MOTIVATION

Pricing is per input/output token

- Limited context size
- LLM gets **distracted** by **useless** information
- **Pricing** is **per input/output token**



GPT-5 Default

The best model for coding and agentic tasks across domains

Compare

Try in Playground

REASONING



SPEED



PRICE

\$1.25 · \$10

INPUT

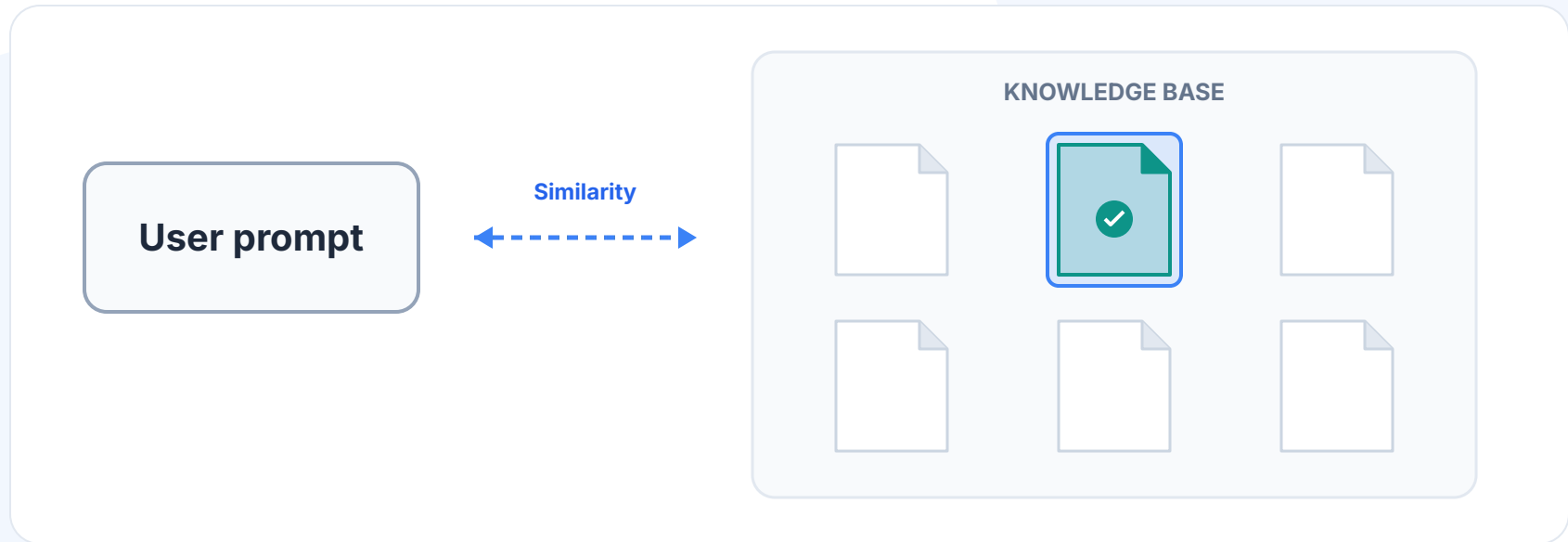


OUTPUT



RETRIEVE, GENERATE, AUGMENT

1. **Retrieve** relevant document via similarity operation across the knowledge base



RETRIEVE, GENERATE, AUGMENT

2. **Augment** prompt with retrieved information

User prompt

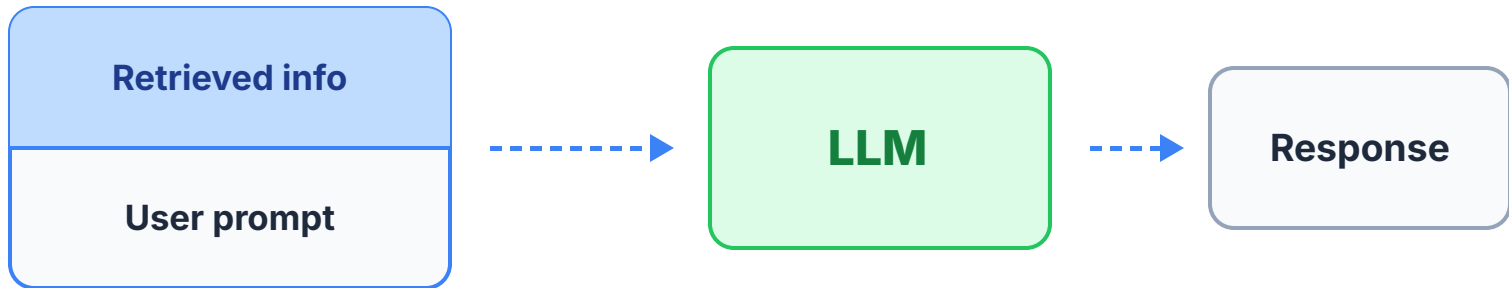


Retrieved info

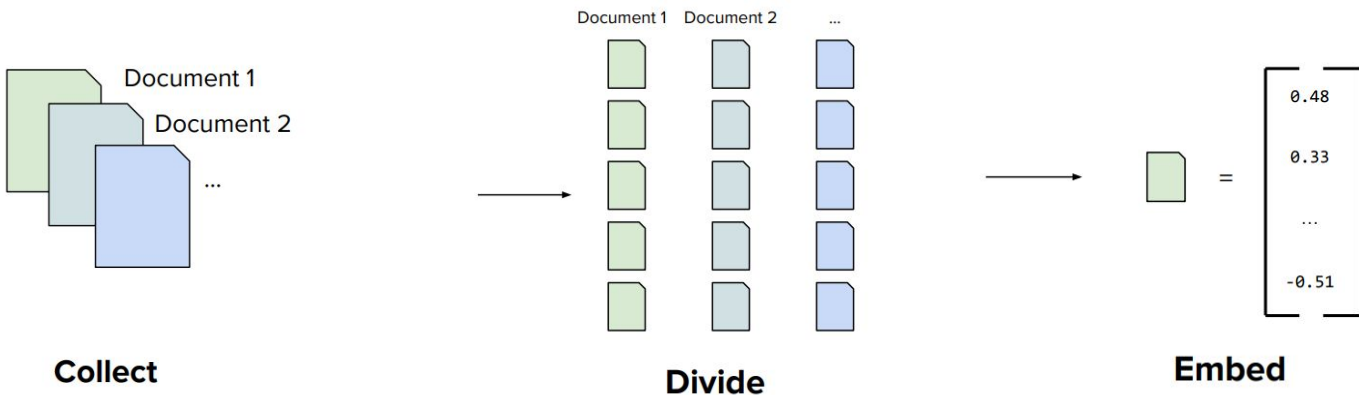
User prompt

RETRIEVE, GENERATE, AUGMENT

3. Generate response



Prerequisite: Create knowledge base



Hyperparameters: Embedding size • Chunk size • Overlap between chunks

1. Candidate retrieval

Objective: Select potentially-relevant candidates from the knowledge base



Maximize Recall

Ensure the retrieval step captures all potentially relevant context chunks to avoid missing critical information before scoring.



Retrieval Methods

- **Semantic Embeddings:** Dense vector similarity search.
- **Keyword-Based:** Optional sparse lexical matching.

Retrieval Overview

STEP 1



Candidate Retrieval

Select potentially-relevant candidates from the knowledge base.

- **Maximize recall** to capture all relevant contexts.
- **Semantic embeddings** & optional keyword methods.

STEP 2



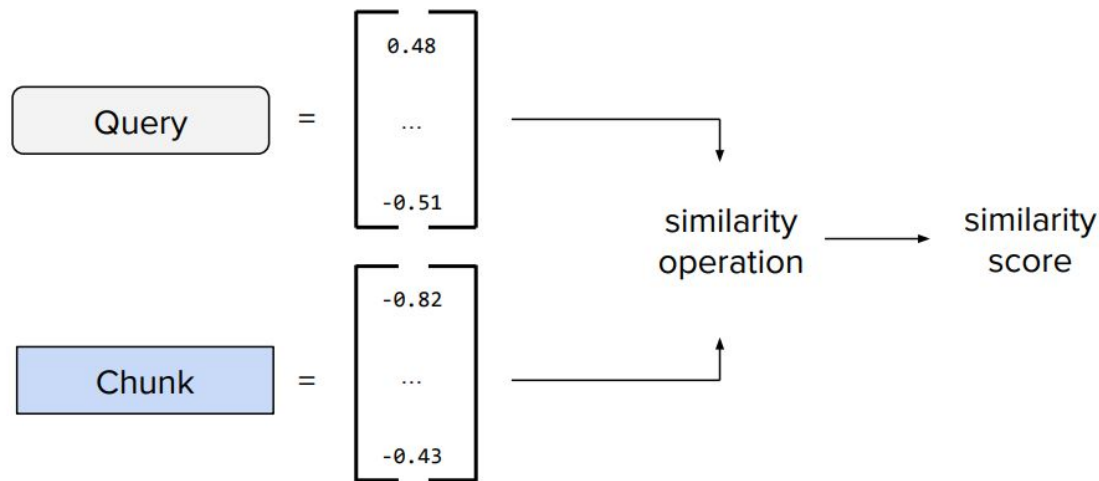
Ranking

Give final relevance score to prioritize top context.

- **Maximize precision** for precise context injection.
- **Re-ranking** on a smaller set of candidate documents.

Objective. Select potentially-relevant candidates via search across knowledge base

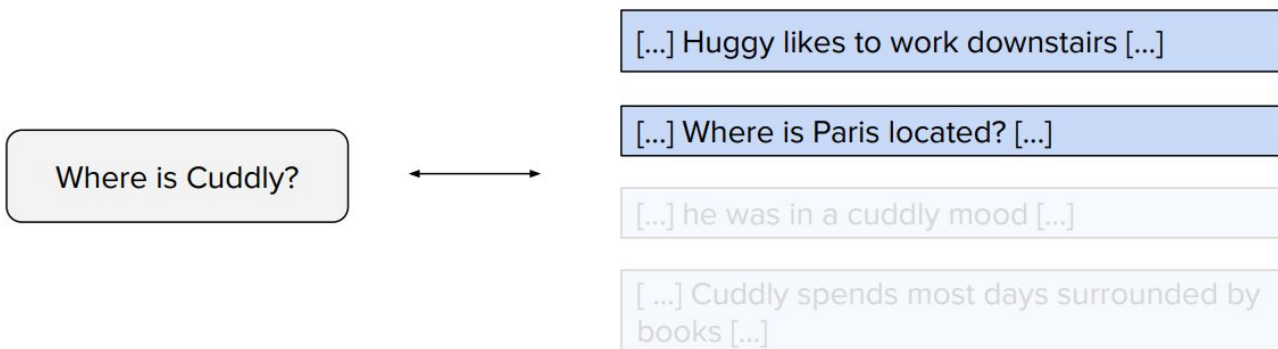
Method 1. Semantic search using **embeddings-based similarity**



Step 1. Candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

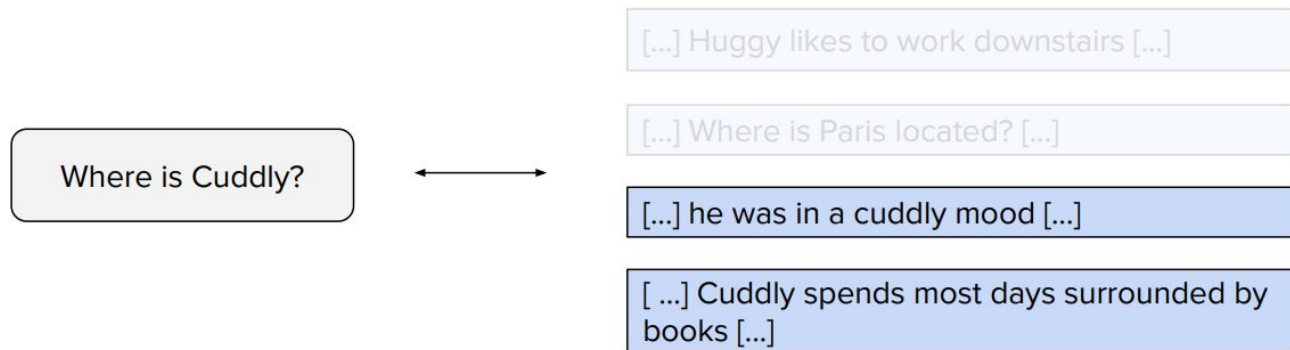
Method 1. Semantic search using **embeddings-based similarity**



Step 1. Candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

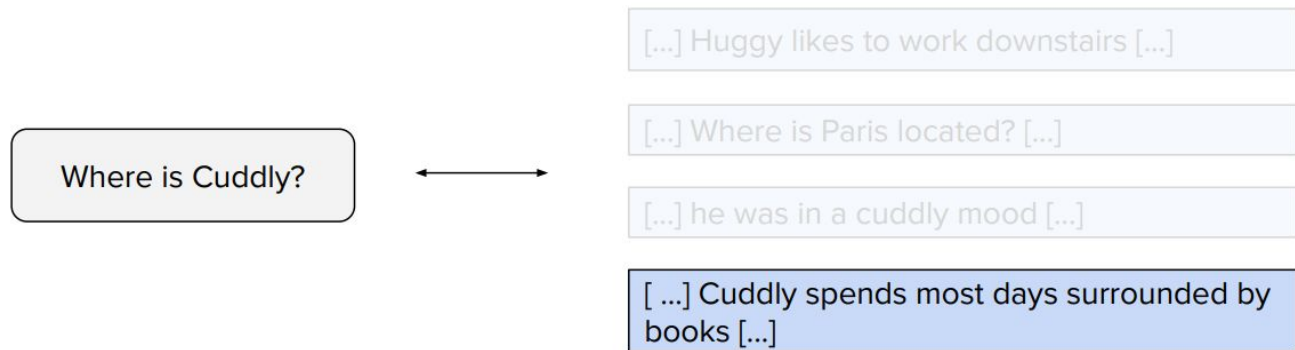
Method 2. Search based on keywords matching via **BM25**



Step 1. Candidate retrieval

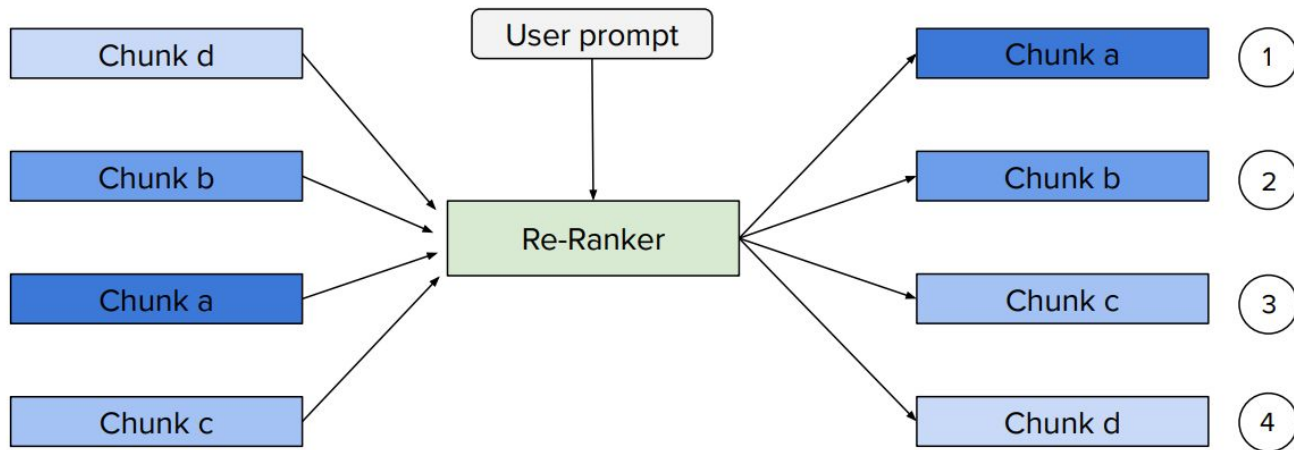
Objective. Select potentially-relevant candidates via search across knowledge base

Method 3. Search based on **hybrid combination** of semantics and BM25



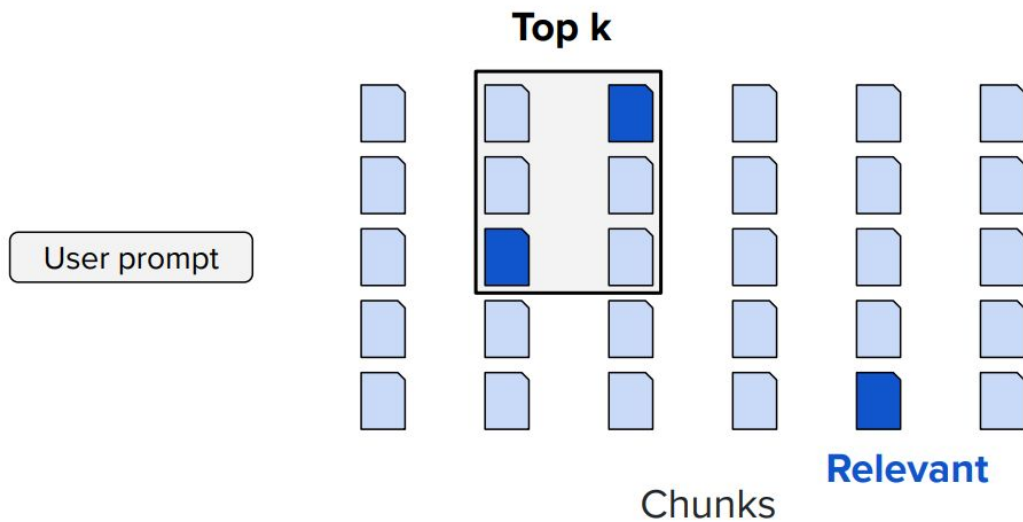
Step 2. Ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker



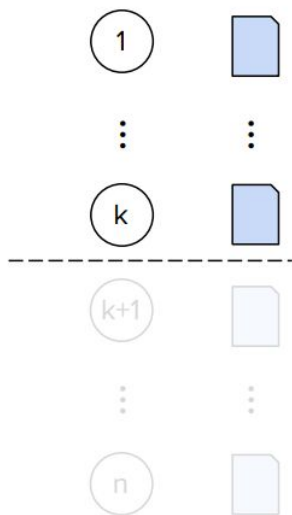
Quantify performance of **retrieval**

Setup. Evaluate if **retrieved chunks** are **relevant**



Quantify performance of retrieval

Ranking



$$\text{Recall}@k = \frac{|\text{relevant in top } k|}{|\text{relevant}|}$$

- Reciprocal Rank at k (RR@k)
- Normalized Discounted Cumulative Gain at k (NDCG@k)
- Precision at k

Overview of Tool Calling

Enabling autonomous LLMs to interact with external systems, APIs, and data sources



1. Schema Definition

Developers define clear JSON schemas for external functions, endpoints, and tools.



2. Model Decision

The LLM evaluates user intent and dynamically chooses which tool to invoke with arguments.



3. Execution & Response

System executes the call, feeds output back to the LLM to complete user requests autonomously.

Definition of Tool Calling

”

"**Tool calling** [...] allows autonomous systems to complete complex tasks by dynamically accessing and **[may act]** upon external resources."

Real-life Example: Without vs. With Tools

LLM as we've known it so far

Find a bear near me!

LLM

Sorry, I don't know which bears are near you.

LLM with tools

<function API>

Find a bear near me!

LLM

?

Real-life Example: `find_teddy_bear.py`

`find_teddy_bear.py`

```
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
        location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```
# Extract coordinates from API response
bear_lat, bear_lon = closest_teddy_bear["coords"]

# Compute distance to the bear using geopy (returns distance in meters)
distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

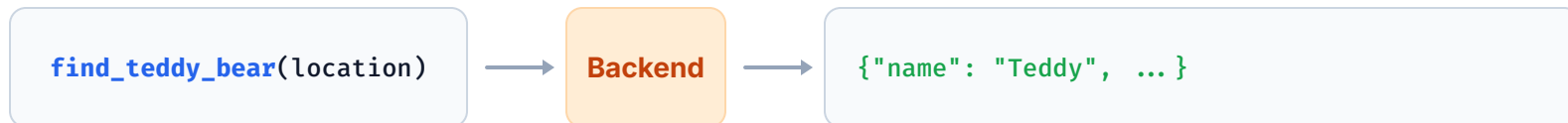
return TeddyBearInfo(
    name=closest_teddy_bear["name"],
    distance_meters=round(distance, 2),
    mood=closest_teddy_bear["mood"],
    message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
            f"and only {round(distance, 2)} meters away!"
)
```

How Function Calling Works

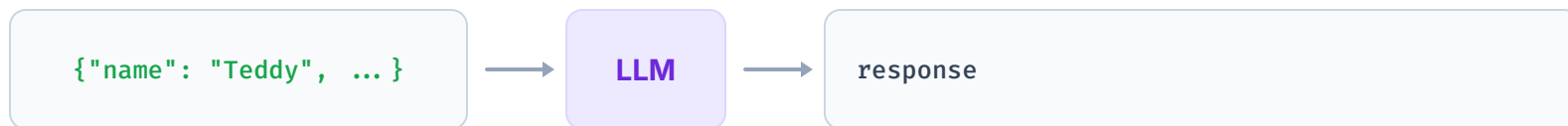
1. Let LLM find **argument** for relevant **function** call



2. Make **function** call

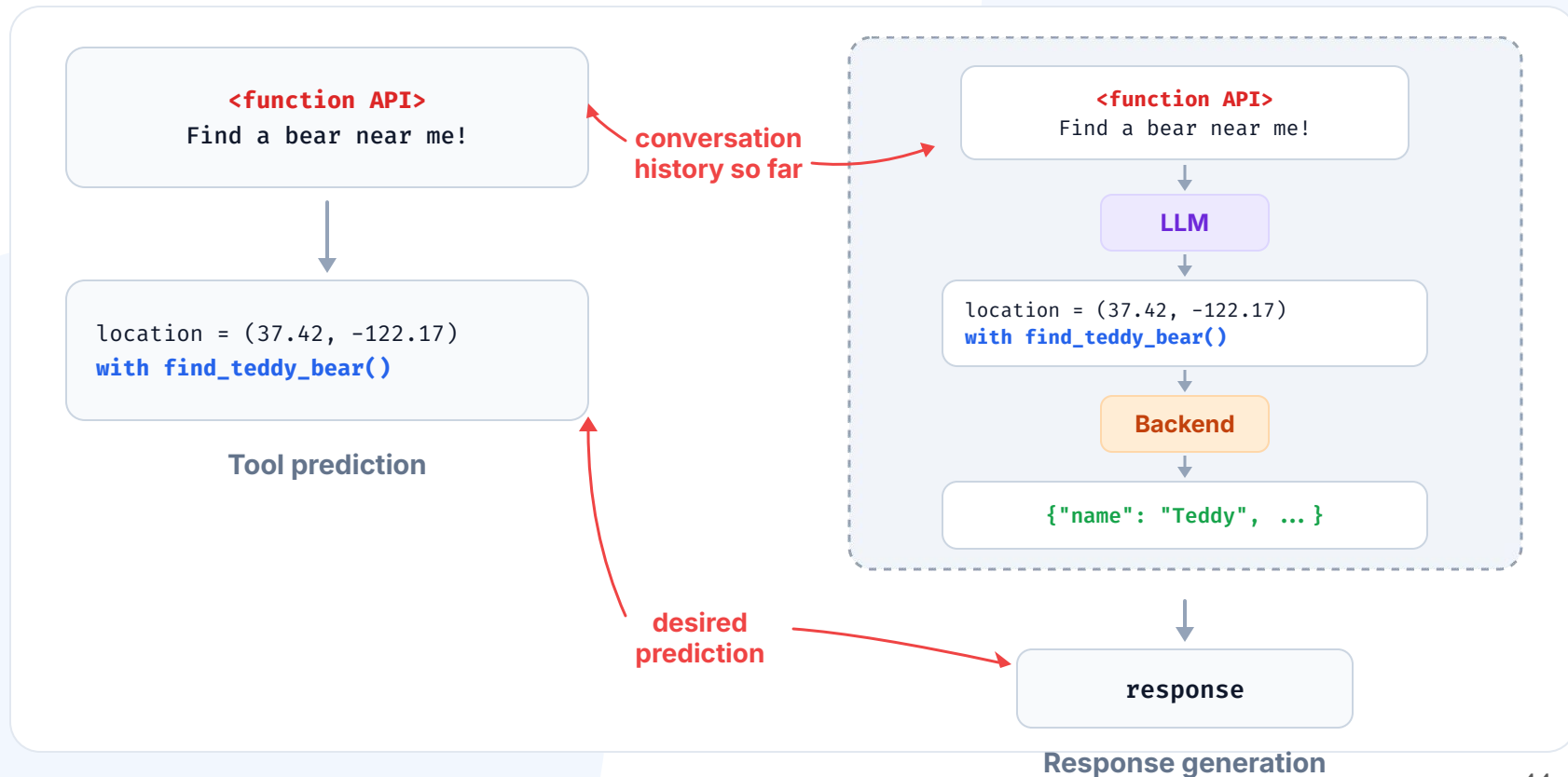


3. Let LLM deduce conclusion **based on results**



Teach a Model to Use a Tool

Method 1: via training



Teach a Model to Use a Tool

Method 2: via prompting

how to write such a description?

`<function API>`

+

`<detailed explanation on how to use it>`

Find a bear near me!

Examples of Common Use Cases

Information

- Web/database search
- Weather, stocks, and any other tracker
- Codebase analysis

Computation

- Calculator
- Code execution (often in Python)

Action

- Send emails / messages
- In-computer action
- Assistant domain tasks

...and many more!

In Practice: Many Tools

```
def find_teddy_bear(  
    location: tuple[float,  
                    float]  
) -> TeddyBearInfo:  
    # ...
```

```
def hug_teddy_bear(  
    recipient: TeddyBear,  
    intensity: str = "warm"  
) -> HugResponse:  
    # ...
```

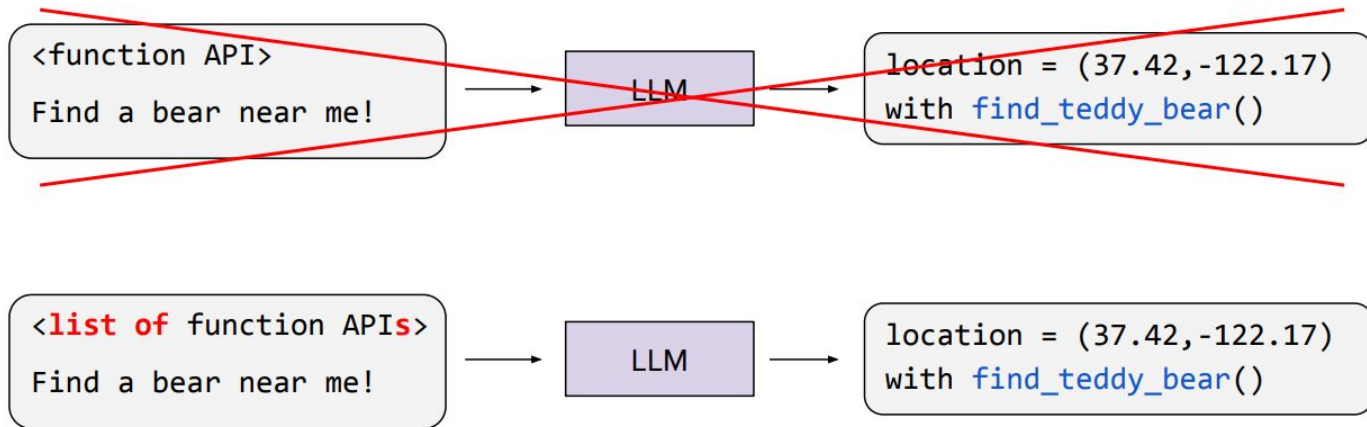
```
def check_teddy_mood(  
    name: TeddyBear  
) -> TeddyMood:  
    # ...
```

```
def send_teddy_gift(  
    recipient: TeddyBear,  
    gift: str = "poetry book"  
) -> GiftResponse:  
    # ...
```

```
def schedule_playdate(  
    host: TeddyBear,  
    guest: TeddyBear,  
    time: datetime  
) -> Confirmation:  
    # ...
```

```
def send_message(  
    recipient: TeddyBear,  
    message: str  
) -> MessageResponse:  
    # ...
```

In Practice: Many Tools



Tools Summary: Benefits



Enhanced Utility

LLMs just became way more useful by seamlessly integrating with specialized tools and software workflows.



Real-World Actions

They can directly interact with the real world through APIs, executing functions and side effects safely.



Bypass Cutoffs

Overcomes static "knowledge cutoff" limitations by retrieving live, up-to-date data on demand.

Tools Summary: Challenges



Reduced Performance

More tools = decrease performance as the model faces greater search space and selection overhead.



Context Limits

Finite context length: not scalable when trying to inject dozens or hundreds of schema definitions.

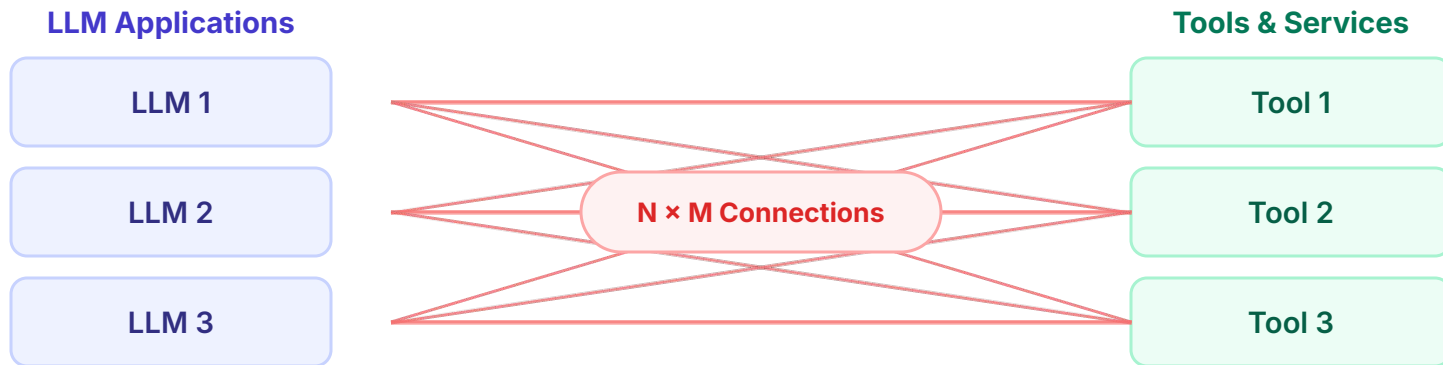


High Overhead

Many tools to define. Lots of work required to build, test, and maintain custom integration code.

Goal: Avoid Duplication of Tool Implementations

Without Standard Protocol: Custom Point-to-Point Integrations ($N \times M$ Complexity)

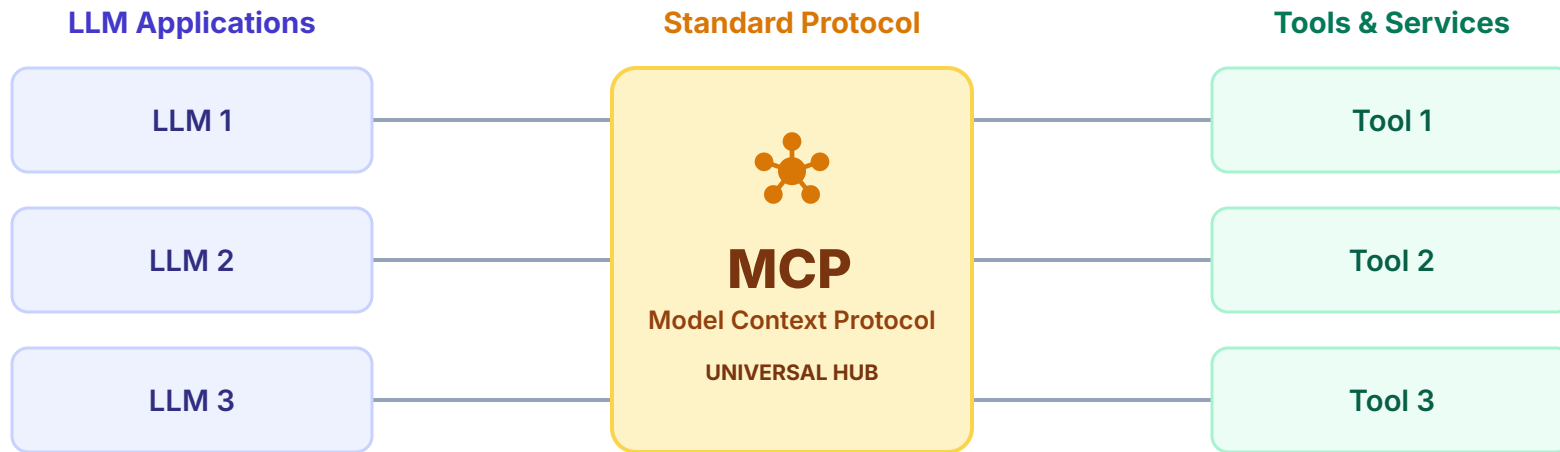


High Overhead & Duplicated Effort

Adding 1 new tool requires custom code for all N models. Adding 1 new model requires integration with all M tools.

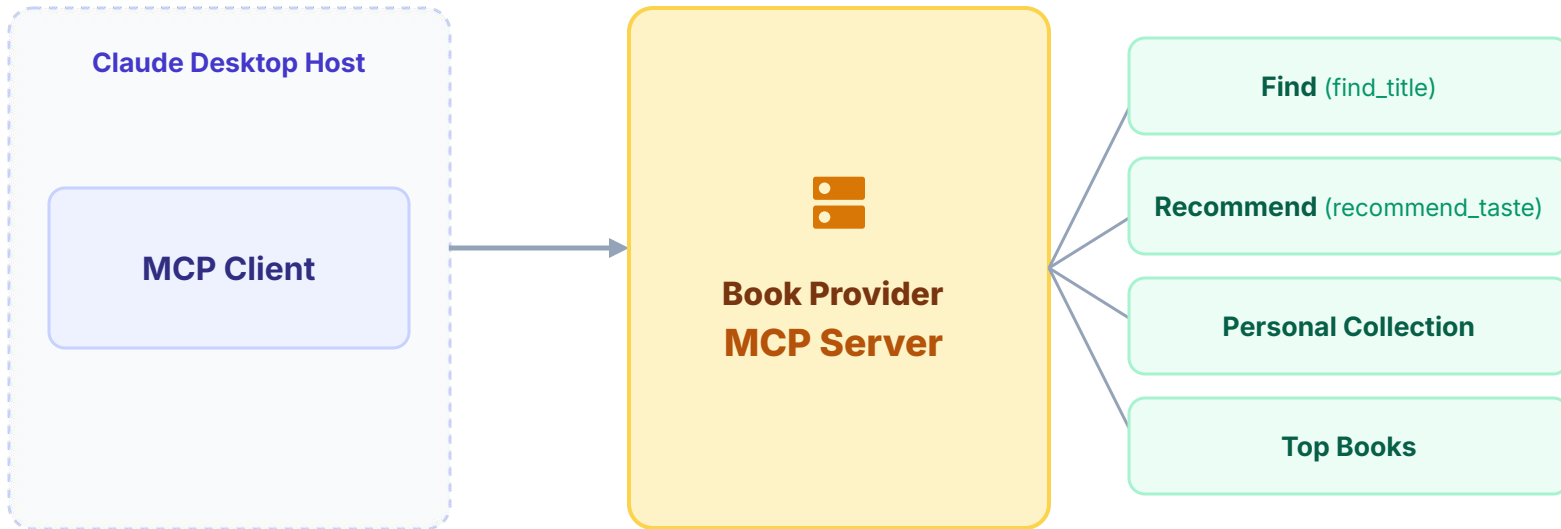
MCP = Model Context Protocol

Idea: Connect tools and data to LLMs in a standard way



Example: MCP Client-Server Request Flow

User Query: "Recommend a new poetry book to my teddy bear"



AI Agents

Autonomous systems capable of perception, reasoning, decision-making, and taking action.



1. Perception

Receives input environment state, user prompts, memory context, and multi-modal observations.



2. Reasoning

Decomposes goals into actionable sub-tasks, evaluates choices, and formulates step-by-step plans.



3. Execution

Executes operations via external tools, APIs, code environments, and specialized MCP servers.

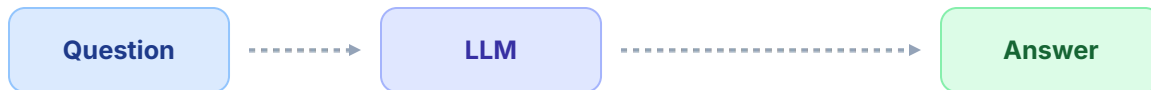
"An **agent** is a system that autonomously pursues goals and completes tasks on a user's behalf."

Evolution of LLM Paradigms

Comparing execution models from direct single-shot prompting to autonomous agent loops.

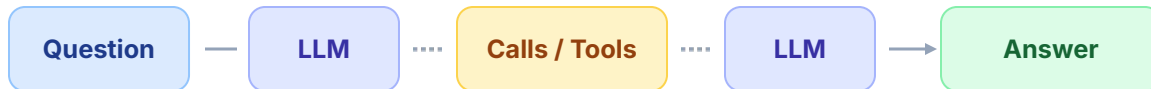
1. Direct Prompting

Single-shot execution



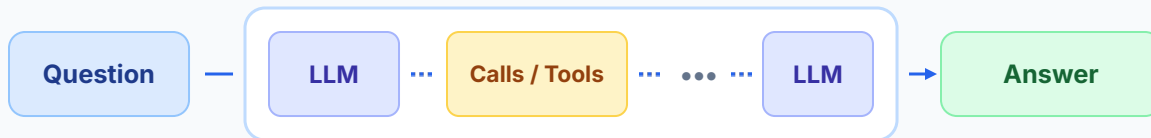
2. Tool-Augmented

Sequential RAG & Tools



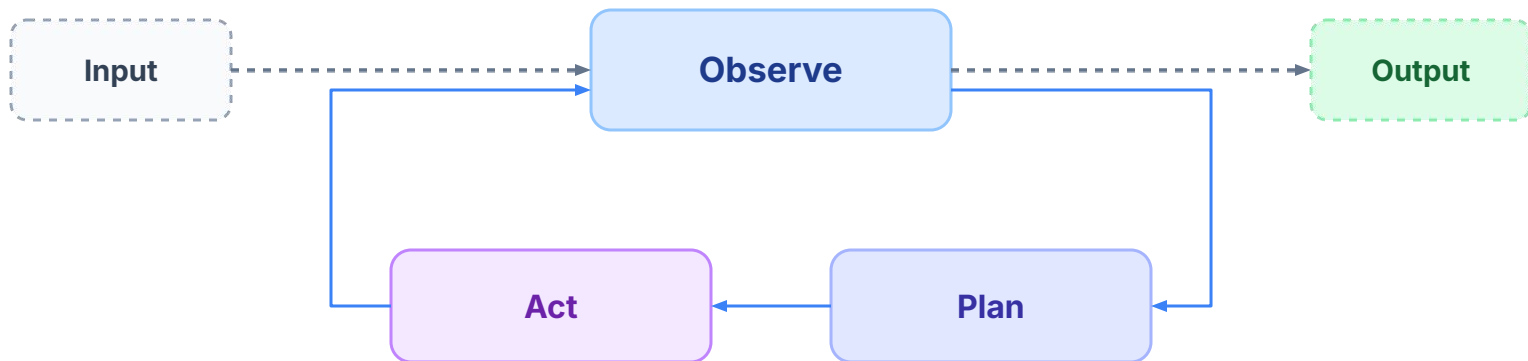
3. Autonomous Agent

Dynamic loop & reasoning



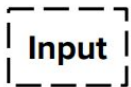
ReAct = Reason + Act

Synergizing reasoning and acting in language models to solve complex tasks.



ReAct in Action

Capturing initial user prompts and external triggers to begin the agent loop.



**My teddy bear is cold.
Please do something.**

Examples:

- Manually entered (e.g. user question)
- External event (e.g. metric going beyc

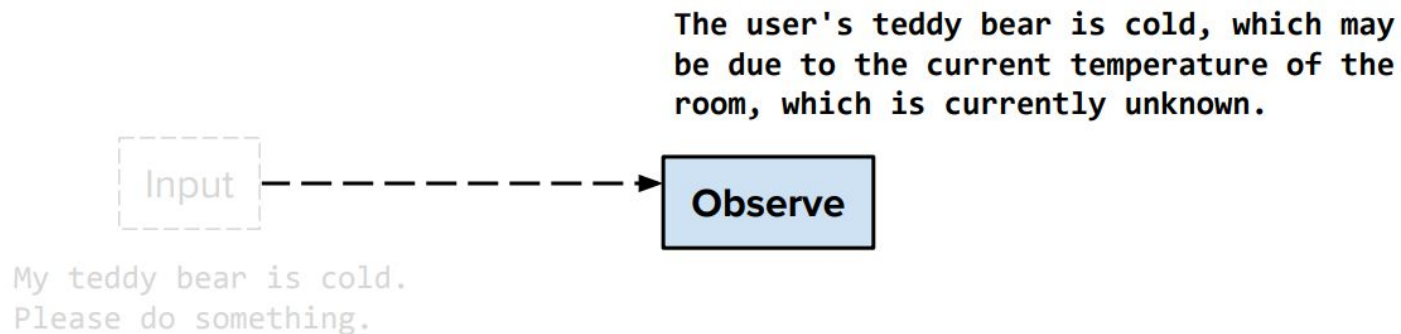
1. Human Prompts

Direct user input, natural language questions, or conversational instructions entered into the agent.

2. External Triggers

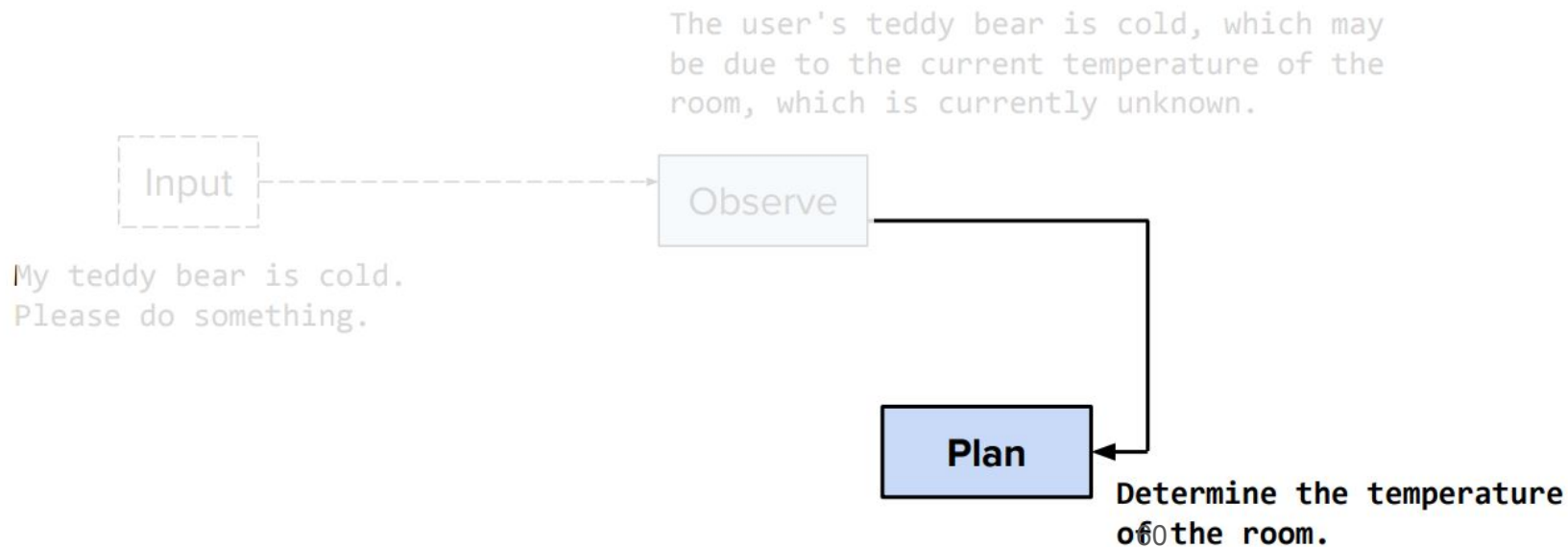
Automated webhooks, scheduled cron tasks, environmental state changes, or API events to begin the loop.

ReAct in action

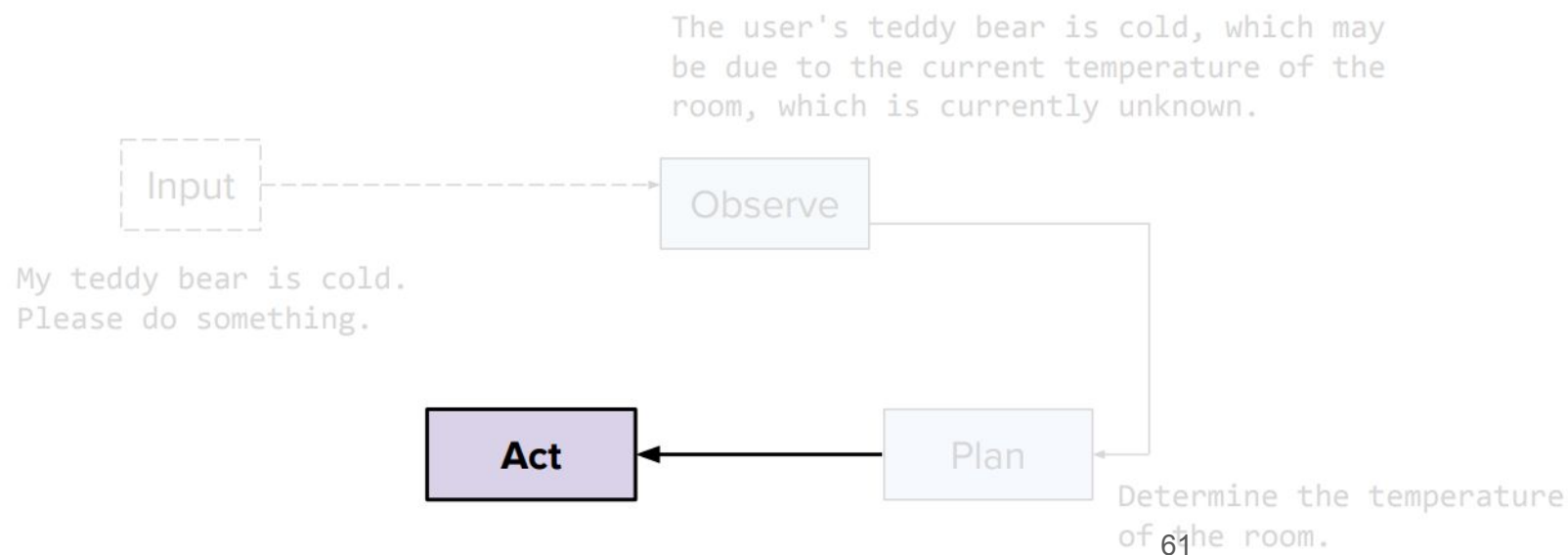


- **Synthesize** previous actions + explicitly **state** what is **currently known** including own knowledge
- **Reasoning**-heavy step to figure out **what is needed**

ReAct in action

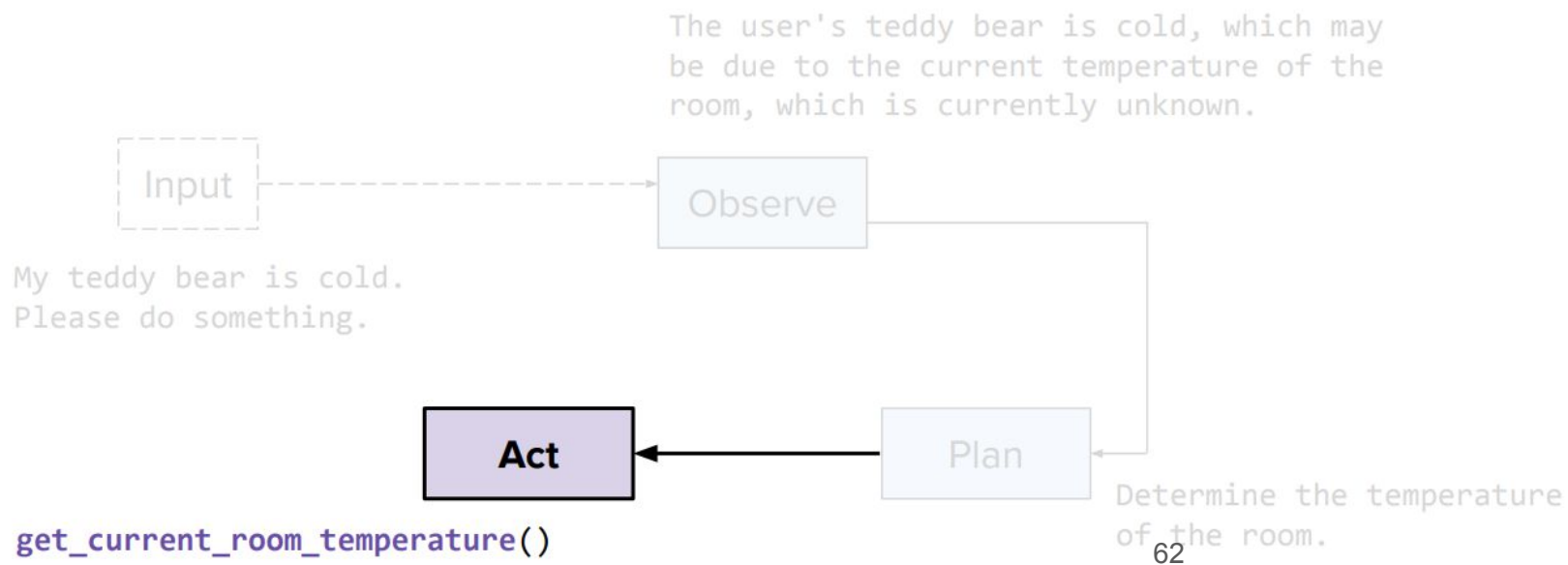


Detail **what tasks** need to be accomplished and **what tools** to call



- Perform an action via an **API**
- **Look** for info in a **database** of documents

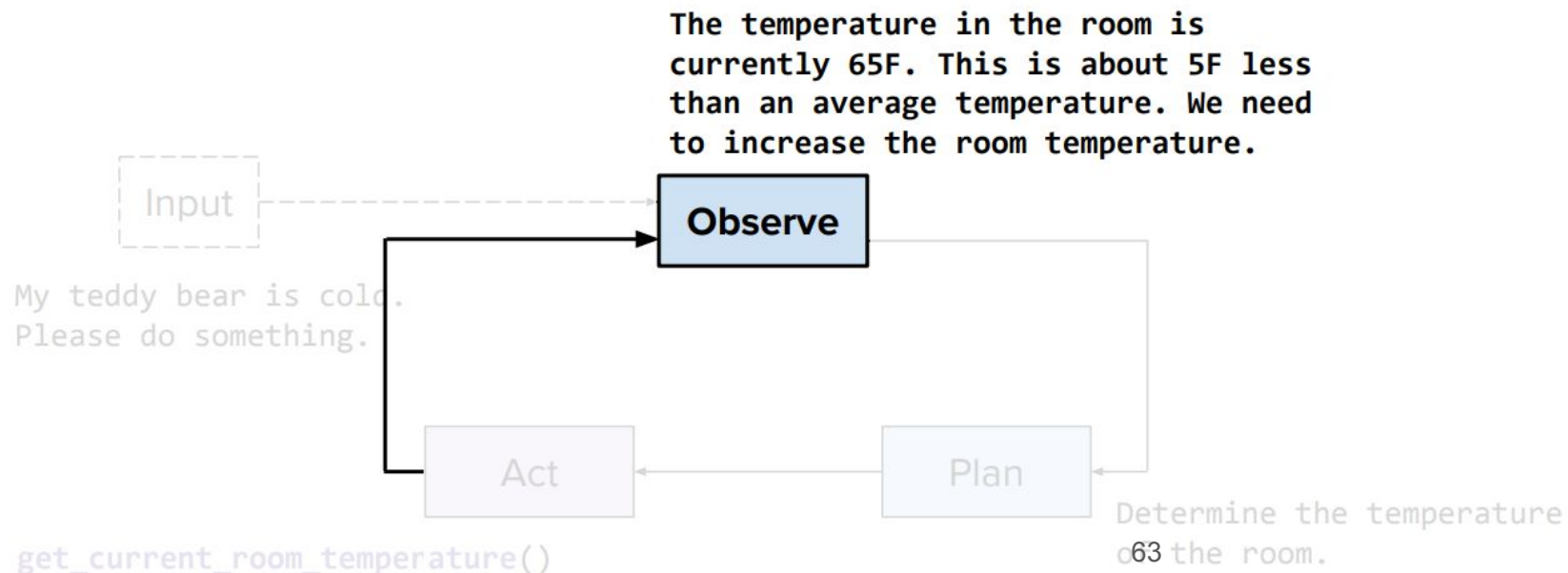
ReAct in action



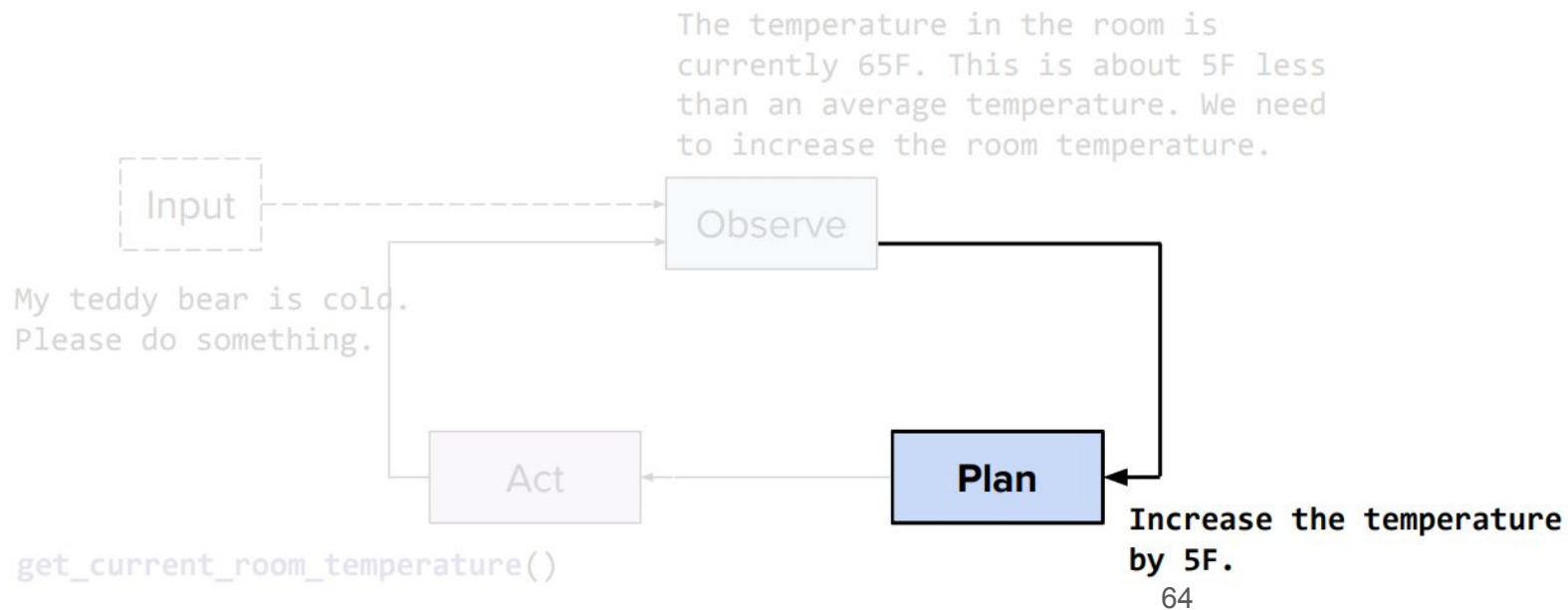
62

- Perform an action via an **API**
- **Look** for info in a **database** of documents

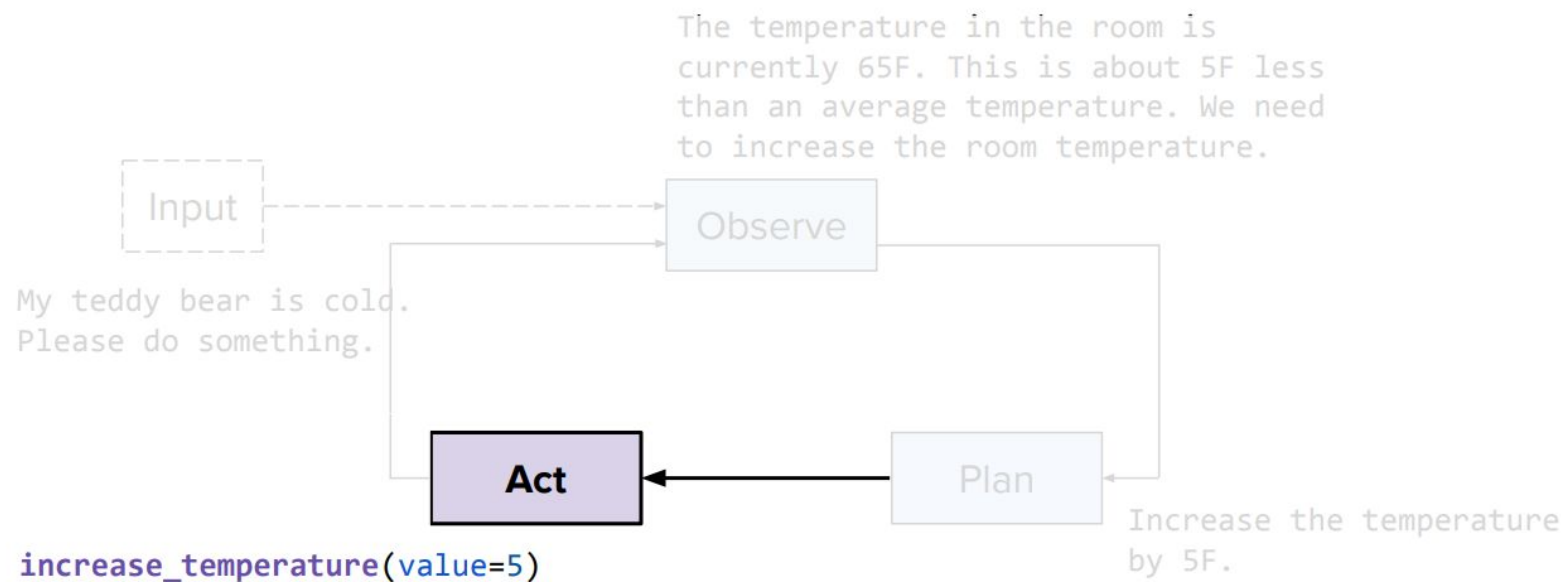
ReAct in action



ReAct in action



ReAct in action

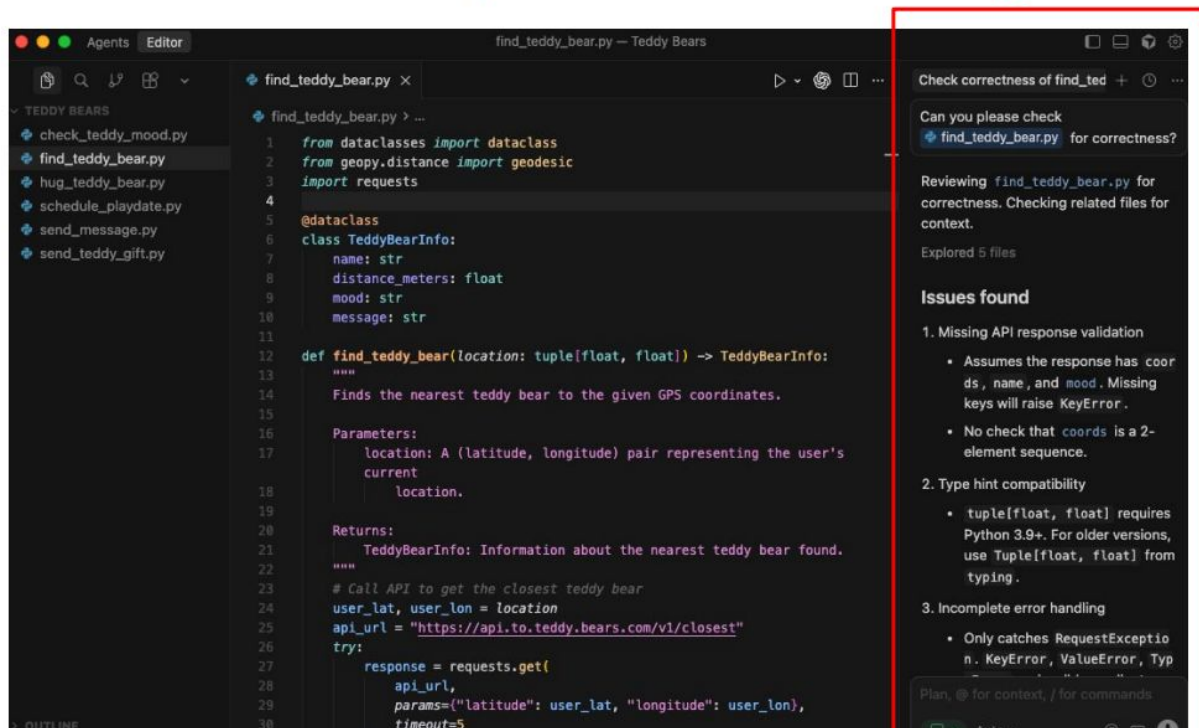


"Agentic" View



Personal favorite use case: coding!

Personal favorite use case: coding!



The screenshot displays an AI agent interface with a code editor and a chat window. The code editor shows the file `find_teddy_bear.py` with the following content:

```
1 from dataclasses import dataclass
2 from geopy.distance import geodesic
3 import requests
4
5 @dataclass
6 class TeddyBearInfo:
7     name: str
8     distance_meters: float
9     mood: str
10    message: str
11
12 def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
13     """
14     Finds the nearest teddy bear to the given GPS coordinates.
15
16     Parameters:
17         location: A (latitude, longitude) pair representing the user's
18             current location.
19
20     Returns:
21         TeddyBearInfo: Information about the nearest teddy bear found.
22     """
23     # Call API to get the closest teddy bear
24     user_lat, user_lon = location
25     api_url = "https://api.to.teddy.bears.com/v1/closest"
26     try:
27         response = requests.get(
28             api_url,
29             params={"latitude": user_lat, "longitude": user_lon},
30             timeout=5
```

The chat window on the right, titled "Check correctness of find_ted", contains the following text:

Can you please check `find_teddy_bear.py` for correctness?

Reviewing `find_teddy_bear.py` for correctness. Checking related files for context.

Explored 5 files

Issues found

1. Missing API response validation
 - Assumes the response has `coords`, `name`, and `mood`. Missing keys will raise `KeyError`.
 - No check that `coords` is a 2-element sequence.
2. Type hint compatibility
 - `tuple[float, float]` requires Python 3.9+. For older versions, use `Tuple[float, float]` from `typing`.
3. Incomplete error handling
 - Only catches `RequestException`. `KeyError`, `ValueError`, `Type`

At the bottom of the chat window, there is a prompt: "Plan, @ for context, / for commands".

A red arrow points from the word "agent" to the chat window.

LLM & AI Agents Lecture

Thank You!

Questions & Discussion